PARADIGMA FUNCIONAL EM HASKELL

Sthefany Moura Godinho
sthefanymouragodinho@gmail.com

Resumo

Este relatório apresenta o desenvolvimento de atividades práticas utilizando a linguagem de programação Haskell, com base nos conceitos do paradigma funcional. O objetivo é aplicar os conhecimentos estudados na disciplina de Programação Funcional, por meio da implementação de funções para resolver diferentes problemas. As atividades abordam conceitos como recursividade, manipulação de listas, strings, tuplas, estruturas condicionais e avaliação preguiçosa. Para cada questão, foram desenvolvidas funções utilizando principalmente recursividade, além de recursos de entrada e saída de dados para facilitar a execução dos programas. As soluções foram testadas no interpretador GHCi, permitindo verificar os resultados obtidos. Dessa forma, o trabalho busca demonstrar a aplicação dos conceitos estudados e contribuir para a compreensão da programação funcional em Haskell.

1 Introdução

A programação funcional é um paradigma de programação baseado na utilização de funções para realizar operações e resolver problemas computacionais. Diferentemente da programação imperativa, que utiliza principalmente comandos e alterações de variáveis, a programação funcional prioriza a avaliação de expressões e a utilização de funções para transformar dados de entrada em resultados.

A linguagem Haskell é utilizada no estudo desse paradigma, permitindo trabalhar com conceitos como funções puras, recursividade, tipos de dados, listas e tuplas. Uma das características importantes dessa linguagem é a possibilidade de construir soluções por meio de funções que podem chamar a si mesmas, utilizando condições de parada para encerrar a execução.

Neste trabalho, são apresentadas as resoluções de dez questões propostas na Segunda Avaliação da disciplina de Programação Funcional. As atividades envolvem operações matemáticas, manipulação de listas e strings, geração de sequências numéricas e processamento de informações armazenadas em tuplas. Também são utilizados conceitos de listas infinitas e avaliação preguiçosa, que permitem trabalhar com sequências cujos elementos são calculados conforme a necessidade.

As soluções foram desenvolvidas utilizando principalmente funções recursivas, conforme as orientações da atividade, e testadas no interpretador GHCi. O relatório apresenta uma explicação sobre o funcionamento das funções e exemplos de execução, demonstrando os resultados obtidos em cada questão.

2 Questão 1 – Somatório

A função `somatorio` recebe um número inteiro como parâmetro e calcula a soma dos números de 1 até o valor informado. Para realizar esse cálculo, utiliza recursividade, somando o número atual com o resultado da chamada da função para o número anterior.

O caso base ocorre quando o número é igual a zero, retornando 0 e encerrando a recursão. Caso contrário, a função continua diminuindo o número em uma unidade.

A função `main` utiliza `getLine` para receber um número digitado pelo usuário. Em seguida, o valor é convertido para o tipo `Int` utilizando `read` e enviado para a função `somatorio`. O resultado é exibido na tela utilizando `putStrLn`.

2.1 Execução

Entrada	Resultado
3	6
5	15
10	55

3 Questão 2 – Máximo Divisor Comum

A função `mdc` recebe dois números inteiros e calcula o Máximo Divisor Comum utilizando o algoritmo de Euclides. Esse algoritmo utiliza o resto da divisão entre os dois números para encontrar o resultado.

A função verifica inicialmente se o segundo número é igual a zero. Quando essa condição é verdadeira, retorna o valor absoluto do primeiro número, utilizando `abs`.

Caso contrário, a função realiza uma chamada recursiva, passando o segundo número como primeiro parâmetro e o resto da divisão como segundo parâmetro. Esse processo continua até que o segundo número seja zero.

Na função `main`, o usuário informa dois números. Os valores são lidos com `getLine`, convertidos para `Int` e enviados para a função `mdc`. Em seguida, o resultado é apresentado na tela.

3.1 Execução

Número 1	Número 2	MDC
48	18	6
20	10	10
15	25	5

4 Questão 3 – Múltiplos de 3

A função `multiplos` recebe uma lista de números inteiros e um número utilizado como limite. Seu objetivo é retornar uma nova lista contendo apenas os valores que são múltiplos de 3 e maiores que o limite informado.

A função utiliza recursividade para percorrer a lista, separando o primeiro elemento dos demais. Para cada elemento, verifica se o resto da divisão por 3 é igual a zero e se o valor é maior que o número informado.

Quando as duas condições são verdadeiras, o elemento é adicionado à lista resultante utilizando o operador `:`. Caso contrário, a função continua a verificação dos elementos restantes. A recursão termina quando a lista fica vazia.

Na função `main`, o usuário informa a lista e o número limite. Os dados são convertidos para os tipos correspondentes e enviados para a função `multiplos`. O resultado é exibido na tela.

4.1 Execução

Lista	Limite	Resultado
[3,6,9,12,15,20]	7	[9,12,15]
[1,3,6,8,12]	3	[6,12]
[2,4,5,7]	2	[]

5 Questão 4 – Elementos em Comum

A função `comuns` recebe duas listas de números inteiros ordenadas e retorna uma terceira lista contendo os elementos presentes nas duas listas, mantendo a ordem e evitando valores repetidos.

Para realizar a comparação, a função separa o primeiro elemento de cada lista e verifica se eles são iguais. Quando os valores são iguais, o elemento é adicionado ao resultado. Em seguida, a função auxiliar `removerIguais` é utilizada para ignorar possíveis repetições desses valores.

Quando os elementos são diferentes, a função avança na lista que possui o menor valor. Esse processo continua recursivamente até que uma das listas fique vazia, encerrando a execução.

A função `main` recebe as duas listas informadas pelo usuário, realiza a conversão dos dados e chama a função `comuns`. A lista resultante é apresentada na tela.

5.1 Execução

Lista 1	Lista 2	Resultado
[1,2,2,3,5,7]	[2,2,3,4,7]	[2,3,7]
[1,3,5]	[2,4,6]	[]
[1,2,3]	[2,3,4]	[2,3]

6 Questão 5 – União de Listas

A função `uniao` recebe duas listas de números inteiros e retorna uma nova lista contendo os elementos das duas, sem valores repetidos.

Inicialmente, as listas são concatenadas utilizando o operador `++`. Em seguida, a lista resultante é enviada para a função auxiliar `semRepetidos`.

A função `semRepetidos` percorre a lista recursivamente, mantendo o primeiro elemento e utilizando a função `remover` para eliminar as outras ocorrências desse mesmo valor.

A função `remover` compara cada elemento com o número informado. Quando os valores são iguais, o elemento é ignorado. Caso contrário, ele é mantido na lista. A recursão termina quando a lista fica vazia.

Na função `main`, o usuário informa duas listas, que são convertidas para o tipo `[Int]`. Depois, a função `uniao` é chamada e o resultado é exibido na tela.

6.1 Execução

Lista 1	**Lista 2**	**Resultado**
[1,2,3,3]	[3,4,5,1]	[1,2,3,4,5]
[1,1,2]	[2,3,3]	[1,2,3]
[4,5]	[6,7]	[4,5,6,7]

7 Questão 6 – Inversão de String

A função `inverter` recebe uma string e retorna seus caracteres em ordem inversa. Como em Haskell uma string é uma lista de caracteres, a função utiliza recursividade para realizar a inversão.

Inicialmente, verifica se a string está vazia. Nesse caso, retorna uma lista vazia, encerrando a recursão.

Caso contrário, a função separa o primeiro caractere do restante da string. Em seguida, realiza uma chamada recursiva para inverter os caracteres restantes e utiliza o operador `++` para adicionar o primeiro caractere ao final.

Na função `main`, o usuário informa uma palavra por meio de `getLine`. O texto é enviado para a função `inverter`, e o resultado é apresentado utilizando `putStrLn`.

7.1 Execução

Entrada	**Resultado**
casa	asac
Haskell	lleksaH
amor	roma

8 Questão 7 – String de Maior Comprimento

A função `maiorString` recebe duas strings e retorna aquela que possui a maior quantidade de caracteres. Quando as duas strings possuem o mesmo comprimento, a função retorna a primeira.

Para realizar a comparação, foi criada a função auxiliar `tamanho`, que utiliza recursividade para contar os caracteres de uma string.

A função `tamanho` verifica se a string está vazia. Quando isso acontece, retorna zero. Caso contrário, soma 1 e realiza uma chamada recursiva para os caracteres restantes.

A função `maiorString` utiliza os resultados de `tamanho` para comparar os textos. A condição de maior ou igual garante que, em caso de empate, seja retornado o primeiro texto.

Na função `main`, o usuário informa duas strings, que são enviadas para a função `maiorString`. O resultado é exibido na tela utilizando `putStrLn`.

8.1 Execução

Texto 1	Texto 2	Resultado
Ana	Carlos	Carlos
Casa	Gato	Casa
Python	Haskell	Haskell

9 Questão 8 – Tabuada com Tuplas

A função `tabuada` recebe um número inteiro e retorna uma lista de triplas contendo a tabuada desse número de 1 até 10.

Cada tripla possui três valores: o número informado, o contador da multiplicação e o resultado do cálculo.

Para construir a lista, a função `tabuada` chama a função auxiliar `gerarTabuada`, iniciando o contador com o valor 1.

A função `gerarTabuada` utiliza recursividade para gerar cada tripla e aumentar o contador em uma unidade. Quando o contador ultrapassa 10, a função retorna uma lista vazia, encerrando a recursão.

Na função `main`, o usuário informa um número inteiro, que é convertido utilizando `read`. Em seguida, a função `tabuada` é chamada e a lista de triplas é exibida na tela.

9.1 Execução

Para a entrada 3, a função retorna:

```
[(3,1,3),(3,2,6),(3,3,9),(3,4,12),
 (3,5,15),(3,6,18),(3,7,21),
 (3,8,24),(3,9,27),(3,10,30)]
```

A tabela abaixo apresenta outros exemplos de multiplicações presentes nas listas geradas.

Entrada	Exemplo de tripla
2	(2,5,10)
3	(3,5,15)
5	(5,10,50)

10 Questão 9 – Listas Infinitas

Nesta questão, foram utilizadas listas infinitas e avaliação preguiçosa, também conhecida como *lazy evaluation*. Esse recurso permite trabalhar com sequências infinitas sem precisar calcular todos os seus elementos de uma só vez.

10.1 Sequência de Fibonacci

A lista `fibs` representa uma sequência infinita de Fibonacci, iniciando com os valores 0 e 1. Nessa sequência, cada novo número é obtido pela soma dos dois anteriores.

A função `gerarFibonacci` recebe dois números como parâmetros, representando o valor atual e o próximo valor da sequência.

Em cada chamada, adiciona o valor atual à lista e chama a si mesma, utilizando o próximo valor e a soma dos dois números.

A função `nFibs` recebe a quantidade de termos desejada e chama a função auxiliar `pegar`, responsável por percorrer a lista e retornar somente essa quantidade.

A recursão de `pegar` termina quando a quantidade solicitada chega a zero.

Na função `main`, o usuário informa a quantidade de termos. O valor é convertido para `Int` e enviado para a função `nFibs`. Em seguida, o resultado é apresentado na tela.

Execução

Quantidade	Resultado
3	[0,1,1]
5	[0,1,1,2,3]
10	[0,1,1,2,3,5,8,13,21,34]

10.2 N-ésimo Número Primo

A lista `primos` representa uma sequência infinita contendo apenas números primos, iniciando a verificação pelo número 2.

A função `gerarPrimos` verifica cada número utilizando a função auxiliar `ehPrimo`. Quando o número é primo, ele é adicionado à lista. Caso contrário, a função continua a busca pelo próximo número.

A função `ehPrimo` verifica se o número é menor que 2 e, nesse caso, retorna `False`. Para os demais valores, utiliza a função `verificarDivisor`.

A função `verificarDivisor` testa possíveis divisores do número. Se encontrar uma divisão exata, retorna `False`. Se o divisor multiplicado por ele mesmo ultrapassar o número, retorna `True`, indicando que o número é primo.

A função `nesimoPrimo` recebe a posição desejada e chama a função auxiliar `buscarPosicao`, que percorre a lista de primos recursivamente até encontrar a posição informada.

Na função `main`, o usuário informa uma posição maior ou igual a 1. Essa posição é enviada para `nesimoPrimo`, e o número primo encontrado é exibido na tela.

Execução

Posição	Número primo
1	2
5	11
10	29

10.3 Múltiplo de 7 Terminado em 3

A função `procurarNumero` recebe um número inteiro e retorna o primeiro valor maior que ele que seja múltiplo de 7 e termine com o algarismo 3.

Para realizar a busca, a função `numerosInfinitos` gera uma lista infinita de números consecutivos.

A função `procurarNumero` inicia a lista no valor informado mais 1, garantindo que o resultado seja maior que a entrada.

Em seguida, a função `procurarLista` percorre os elementos e verifica duas condições: se o resto da divisão por 7 é zero e se o resto da divisão por 10 é igual a 3.

Quando as duas condições são verdadeiras, o número é retornado. Caso contrário, a busca continua no restante da lista.

Na função `main`, o usuário informa um número inteiro, que é convertido para `Integer` e enviado para `procurarNumero`. O resultado é exibido na tela.

Execução

Entrada	Resultado
10	63
63	133
150	203

11 Questão 10 – Base de Dados das Seleções

Nesta questão, foi utilizada uma base de dados fornecida pelo professor, contendo informações sobre dez seleções de futebol.

A função `base` recebe um número inteiro que representa o índice da seleção e retorna uma tupla com quatro informações: nome da seleção, continente, quantidade de títulos mundiais e ano do último título.

A partir dessa base, foram desenvolvidas quatro alternativas utilizando recursividade para consultar e processar os dados.

11.1 Quantidade de Seleções Europeias

A função `contarEuropeias` recebe um índice e percorre a base de dados para contar quantas seleções pertencem ao continente europeu.

Em cada chamada recursiva, a função consulta a seleção correspondente ao índice e verifica se o continente é igual a `Europa`.

Quando essa condição é verdadeira, soma 1 ao resultado das próximas chamadas. Caso contrário, continua a contagem sem acrescentar valores.

A recursão termina quando o índice ultrapassa 10, retornando zero.

A função `europeias` inicia a contagem a partir do primeiro registro.

Execução

Consulta	Resultado
Seleções europeias	7

11.2 Campeãs com Último Título Antes de 2000

A função `campeasAntigas` percorre a base de dados e retorna uma lista contendo os nomes das seleções campeãs cujo último título mundial ocorreu antes do ano 2000.

Para isso, verifica se a quantidade de títulos é maior que zero e se o ano do último título é menor que 2000.

Quando as duas condições são verdadeiras, o nome da seleção é adicionado à lista resultante utilizando o operador `:`.

Caso contrário, a função continua a busca sem adicionar a seleção.

A recursão termina quando o índice ultrapassa 10. A função `listaCampeas` inicia a busca a partir do primeiro registro.

Execução

Resultado
["Uruguai","Inglaterra"]

11.3 Seleção com Mais Títulos Mundiais

A função `maiorCampea` percorre os registros da base para identificar a seleção que possui a maior quantidade de títulos mundiais.

A função recebe três parâmetros: o índice atual, o nome da melhor seleção encontrada até o momento e a maior quantidade de títulos.

Em cada chamada recursiva, compara a quantidade de títulos da seleção atual com o maior valor encontrado anteriormente.

Quando encontra uma seleção com mais títulos, atualiza o nome e a quantidade de títulos. Caso contrário, mantém os valores anteriores.

A recursão termina quando o índice ultrapassa 10, retornando o nome da seleção armazenada.

A função `selecaoMaisTitulos` inicia a busca no primeiro registro, utilizando uma string vazia como nome inicial e zero como quantidade de títulos.

Execução

Seleção	Títulos
Brasil	5

11.4 Total de Títulos por Continente

A função `somarTitulos` recebe o nome de um continente e um índice da base de dados. Seu objetivo é calcular a soma dos títulos mundiais das seleções que pertencem ao continente informado.

Em cada chamada, a função consulta o registro correspondente ao índice e verifica se o continente da seleção é igual ao continente procurado.

Quando essa condição é verdadeira, a quantidade de títulos é adicionada ao resultado da chamada recursiva.

Caso contrário, a função continua percorrendo a base sem realizar a soma.

A recursão termina quando o índice ultrapassa 10, retornando zero.

A função `titulosContinentes` utiliza `somarTitulos` para calcular os totais da América do Sul e da Europa. O resultado é organizado em uma lista de tuplas, contendo o nome do continente e a quantidade total de títulos.

Execução

Continente	Títulos
America do Sul	10
Europa	12

O resultado retornado pela função é:

```
[("America do Sul",10),("Europa",12)]
```

11.5 Execução do Programa Principal

A função `main` apresenta um menu com quatro opções, permitindo executar cada uma das alternativas da questão.

O usuário informa a opção desejada utilizando `getLine`. Em seguida, o valor é convertido para `Int` utilizando `read`.

De acordo com a opção escolhida, a função `main` chama a resolução correspondente e apresenta seu resultado na tela.

Caso seja informado um número diferente das opções disponíveis, o programa exibe a mensagem `Opcao invalida`.

Após apresentar o resultado, a execução da função `main` é encerrada. Para escolher outra alternativa, é necessário executar a função novamente.

12 Conclusão

A realização desta atividade permitiu colocar em prática os conhecimentos adquiridos durante as aulas de Programação Funcional, utilizando a linguagem Haskell para resolver diferentes problemas computacionais. Por meio das questões propostas, foi possível trabalhar conceitos como recursividade, manipulação de listas, strings, tuplas e avaliação preguiçosa.

Durante o desenvolvimento das soluções, a recursividade foi um dos principais recursos utilizados, permitindo resolver os problemas por meio de chamadas sucessivas às próprias funções. Além disso, a utilização de funções auxiliares contribuiu para organizar melhor os códigos e facilitar a compreensão de cada resolução.

Os testes realizados no interpretador GHCi permitiram verificar o funcionamento das funções e os resultados obtidos. A implementação de entradas e saídas também facilitou a interação com os programas e a visualização dos resultados.

Portanto, a atividade contribuiu para ampliar a compreensão sobre o paradigma funcional e a linguagem Haskell, reforçando a importância da recursividade e da organização das funções na construção de soluções simples e eficientes.